

Reviewer Pack — Minimal Symplectic Form Rank Correlation with Frege Proof De...

Entry 355034e2870e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 04:07:10 UTC

Minimal Symplectic Form Rank Correlation with Frege Proof Depth

Entry ID: 355034e2870e **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 04:07:10 UTC

1. Conjecture

Field A (mathematical branch): Symplectic Geometry **Field B** (complexity object): Frege Proof Complexity

Statement:

For any given Boolean formula φ , the minimal symplectic form rank ($\text{sfr}(\varphi)$) of its associated symmetric space is linearly correlated with its Frege proof depth ($d(\varphi)$), such that $\text{sfr}(\varphi) = \Theta(d(\varphi))$.

Rationale (proposer's reasoning):

Symplectic geometry provides a geometric framework that could potentially encode non-trivial information about the complexity of computations. The symplectic form rank measures the dimensionality of the associated symmetric space, which might reflect the structural complexity of the formula. If this rank correlates with the proof depth, it suggests a deep connection between geometric structures and computational complexity.

Taxonomy category: symplectic_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 24379acedd57b0d5

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a given Boolean formula φ , if the minimal symplectic form rank ($\text{sfr}(\varphi)$) is within a factor of 2 from its Frege proof depth ($d(\varphi)$), and no seed produces a correlation coefficient below 0.7.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - symplectic geometry AND Frege proof complexity - minimal symplectic form rank AND Frege proof depth-Boolean formula symplectic space AND Frege proof complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_formula(depth):
        if depth == 1:
            return random.choice(['A', 'B', 'C'])
        else:
            op = random.choice(['&', '|'])
            left = generate_formula(depth - 1)
            right = generate_formula(depth - 1)
            return f"({left} {op} {right})"

    def incidence_matrix(formula):
        if formula.isalpha():
            return [[0]]

        op, left, right = formula[1:-2].split()
        left_matrix = incidence_matrix(left)
        right_matrix = incidence_matrix(right)

        n_left = len(left_matrix)
        n_right = len(right_matrix)

        result = [[0] * (n_left + n_right) for _ in range(n_left + n_right)]

        for i in range(n_left):
            for j in range(n_right):
                if op == '&':
                    result[i][j + n_left] = 1
                elif op == '|':
```

```

        result[j][i + n_right] = 1

    return result

def gramian_matrix(matrix):
    n = len(matrix)
    G = [[0] * n for _ in range(n)]

    for i in range(n):
        for j in range(n):
            for k in range(n):
                G[i][j] += matrix[i][k] * matrix[j][k]

    return G

def min_symplectic_form_rank(gramian):
    n = len(gramian)
    rank = 0

    for i in range(n):
        pivot_found = False
        for j in range(i, n):
            if gramian[j][i] != 0:
                # Swap rows to make the pivot non-zero
                gramian[i], gramian[j] = gramian[j], gramian[i]
                pivot_found = True
                break

        if not pivot_found:
            continue

        rank += 1
        for j in range(i + 1, n):
            factor = Fraction(gramian[j][i], gramian[i][i])
            for k in range(n):
                gramian[j][k] -= factor * gramian[i][k]

    return rank

def frege_proof_depth(formula):
    if formula.isalpha():
        return 1
    else:
        op, left, right = formula[1:-2].split()
        return 1 + max(frege_proof_depth(left), frege_proof_depth(right))

depths = [5, 10, 15, 20, 30, 40]
results = []

for depth in depths:
    formula = generate_formula(depth)
    incidence = incidence_matrix(formula)
    gramian = gramian_matrix(incidence)
    sfr = min_symplectic_form_rank(gramian)
    d = frege_proof_depth(formula)

```

```

    if d == 0:
        continue

    results.append((sfr, d))

if not results:
    return {
        "metric_name": "symplectic_form_rank_correlation",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

sfr_values, d_values = zip(*results)
correlation_coefficient = sum(sfr * d for sfr, d in results) / (sum(sfr**2 for sfr in sfr_values) + sum(d**2 for d in d_values))

return {
    "metric_name": "symplectic_form_rank_correlation",
    "metric_value": correlation_coefficient,
    "instances_tested": len(results),
    "n_max": max(depths),
    "conjecture_holds": correlation_coefficient >= 0.7 and all(1 <= sfr / d <= 2 for sfr, d in results),
    "counterexample": "" if correlation_coefficient >= 0.7 else f"correlation_coefficient={correlation_coefficient}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all(result["conjecture_holds"] for result in results):
        mean_value = sum(result["metric_value"] for result in results) / len(results)
        support_fraction = 1.0
        print(f"RESULT: SUPPORTED mean={mean_value} std=0 support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient_below_0.7\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f51c51cb.py", line 137, in <module>
    trial_result = run_trial(seed)
                    ^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f51c51cb.py", line 99, in run_trial
    incidence = incidence_matrix(formula)
                ^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f51c51cb.py", line 34, in incidence_mat
    op, left, right = formula[1:-2].split()
                    ^^^^^^^^^^^^^^^

```

ValueError: too many values to unpack (expected 3)

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Review and debug the test code to ensure it runs successfully and produces the required data for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14094
2	preregistration	ollama_remote	glm4:latest	0	0	9558
3	novelty	ollama_remote	glm4:latest	0	0	10130
4	novelty	ollama_remote	glm4:latest	0	0	15021
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22641
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17072
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	25807
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16223
9	judge	ollama_remote	glm4:latest	0	0	8830

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 139377 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/355034e2870e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/355034e2870e.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/355034e2870e.tar.gz (if generated)